

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)
Факультет физико-технический
Кафедра теоретической физики и компьютерных технологий

КУРСОВОЙ ПРОЕКТ

Применение методов корректировки пространственных координат объекта в AR-приложениях

Работу выполнил _____ Крючков Артур Максимович

Курс 3

Направление подготовки 09.03.02 Информационные системы и технологии

Научный руководитель

канд. биол. наук, доцент _____ Н.Н. Куликова

Нормоконтролер, преподаватель _____ -

Краснодар

2026

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
ВВЕДЕНИЕ	4
1 Теоретические основы позиционирования объектов в дополненной реальности	6
1.1 Принципы работы систем трекинга в мобильных AR-приложениях	6
1.2 Пространственные системы координат: локальное и мировое пространство в AR	7
1.3 Проблемы позиционирования и методы корректировки пространственных координат	9
2 Разработка AR-приложения с применением методов корректировки координат	12
2.1 Выбор и обоснование инструментария.	12
2.2 Настройка системы Image Tracking и архитектура приложения.	12
2.3 Программная реализация трансформации координат (от локальных координат маркера к мировым)	13
2.4 Практическое применение корректировок	15
2.5 Разработка UI-интерфейса в контексте AR-сцены.	16
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21

Перечень сокращений и обозначений

Сокращение	Расшифровка
AR (Augmented Reality)	Дополненная реальность
VR (Virtual Reality)	Виртуальная реальность
MR (Mixed Reality)	Смешанная реальность
XR (Extended Reality)	Расширенная реальность
SDK (Software Development Kit)	Комплект для разработки программного обеспечения
API (Application Programming Interface)	Программный интерфейс приложения
UI (User Interface)	Пользовательский интерфейс
UX (User Experience)	Пользовательский опыт
GPU (Graphics Processing Unit)	Графический процессор
CPU (Central Processing Unit)	Центральный процессор
IMU (Inertial Measurement Unit)	Инерциальный измерительный блок
SLAM (Simultaneous Localization and Mapping)	Одновременная локализация и построение карты
FOV (Field of View)	Поле зрения

ВВЕДЕНИЕ

В последние годы технологии дополненной реальности (AR) перешли из категории экспериментальных разработок в статус повсеместно используемых инструментов. Особенно активно AR-технологии внедряются в сферы образования, культуры и туризма, позволяя «оживлять» исторические памятники, музейные экспонаты и печатные материалы. Современные мобильные устройства обладают достаточной вычислительной мощностью для рендеринга сложной 3D-графики в реальном времени, однако главная техническая сложность кроется не в отрисовке объектов, а в их корректном позиционировании в физическом пространстве.

Актуальность данной курсовой работы обусловлена тем, что стабильность и реалистичность AR-опыта напрямую зависят от точности совмещения виртуальной системы координат с реальным миром. При разработке AR-приложений разработчики регулярно сталкиваются с проблемами некорректного масштабирования, несовпадения мировых осей и пространственного дрейфа моделей. Без применения специфических методов корректировки пространственных координат, преобразования локальных матриц в мировые и программного контроля состояния AR-сессии, создание качественного иммерсивного продукта невозможно.

Целью работы является исследование и практическое применение методов корректировки пространственных координат 3D-объектов при разработке мобильных AR-приложений.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить теоретические основы систем пространственного трекинга и особенности работы с системами координат в дополненной реальности.

2. Проанализировать типичные проблемы позиционирования виртуальных объектов.

3. Разработать прототип мобильного AR-приложения с использованием технологии Image Tracking на базе игрового движка Unreal Engine 5.

4. Применить на практике алгоритмы математической и программной корректировки координат, ориентации и масштаба для корректного отображения 3D-модели исторического памятника поверх физического маркера.

Объектом исследования является процесс позиционирования и визуализации трехмерных объектов в системах мобильной дополненной реальности.

Предметом исследования выступают методы и алгоритмы корректировки пространственных координат, масштаба и вращения виртуальных объектов.

Практическая значимость работы заключается в создании рабочего прототипа мобильного AR-приложения, в котором реализованы надежные методы трансформации координат от 2D-маркера к 3D-сцене с учетом ограничений мобильных платформ.

1 Теоретические основы позиционирования объектов в дополненной реальности

1.1 Принципы работы систем трекинга в мобильных AR-приложениях

Ключевой задачей любой системы дополненной реальности является точное определение местоположения и ориентации камеры мобильного устройства в трехмерном физическом пространстве. Этот процесс непрерывного вычисления координат называется трекингом (от англ. tracking — отслеживание). Трекинг позволяет программному обеспечению понимать, под каким углом и с какого расстояния пользователь смотрит на реальный мир, чтобы корректно наложить на него виртуальные объекты. Системы трекинга вычисляют так называемые 6 степеней свободы, которые включают три координаты перемещения (X , Y , Z) и три угла вращения (тангаж, крен, рыскание).

В современных мобильных AR-приложениях используются два основных подхода к трекингу: безмаркерный и маркерный.

Безмаркерный трекинг базируется на алгоритмах визуально-инерциальной одометрии (VIO) и технологии SLAM (Simultaneous Localization and Mapping). Камера смартфона захватывает видеопоток и выделяет в нем контрастные точки (особенности рельефа, углы мебели, текстуры пола). Параллельно считываются данные с акселерометра и гироскопа. Объединяя эти данные, система в реальном времени строит облако точек и распознает плоскости (горизонтальные и вертикальные). Данный метод универсален, однако он подвержен ошибкам при плохом освещении, на однотонных поверхностях (например, гладкий белый стол) и при резких движениях камеры, что вызывает дрейф виртуальных объектов.

Маркерный трекинг, который является объектом исследования в данной работе, работает по иному принципу. В качестве точки отсчета используется заранее заданное двухмерное изображение. Процесс включает следующие этапы:

Извлечение признаков (Feature Extraction): На этапе компиляции приложения целевое изображение анализируется алгоритмами компьютерного зрения. Система находит на нем высококонтрастные углы и границы, создавая уникальный цифровой «отпечаток» изображения.

Сопоставление (Matching): Во время работы приложения алгоритм ищет совпадения между признаками в реальном видеопотоке и признаками в базе данных маркеров.

Оценка позы (Pose Estimation): При нахождении совпадения система анализирует искажение перспективы изображения в кадре. Зная реальный физический размер маркера (задаваемый разработчиком в физических единицах, например, в сантиметрах), алгоритм вычисляет точное расстояние до маркера и угол наклона камеры.

Маркерный подход обеспечивает абсолютную привязку к физическому объекту (информационной табличке, плакату, картине), что делает его идеальным выбором для музейных и туристических приложений, где требуется высокая стабильность сцены и отсутствие пространственного дрейфа.

1.2 Пространственные системы координат: локальное и мировое пространство в AR

Для корректного размещения виртуальных 3D-моделей поверх реального мира необходимо связать физическое пространство с математическим пространством графического движка. Эта связь осуществляется посредством систем координат. В архитектуре AR-

приложений принято выделять две основные системы: мировую (World Coordinate System) и локальную (Local Coordinate System).

Мировая система координат инициализируется в момент запуска AR-сессии на устройстве. В этот момент движок устанавливает точку $(0, 0, 0)$ в том месте физического пространства, где находилась камера устройства при запуске. Оси ориентации в мировом пространстве обычно выравниваются с учетом гравитации: ось Z всегда направлена строго вверх, а оси X и Y располагаются в горизонтальной плоскости. Все виртуальные объекты в сцене в конечном итоге позиционируются относительно этого глобального нуля.

Локальная система координат привязывается к конкретному физическому объекту в реальном мире. В случае использования маркерного трекинга, центр распознанного изображения становится началом координат локального пространства $(0, 0, 0)$. Оси в локальном пространстве маркера имеют строгую ориентацию относительно самой картинка:

Оси X и Y лежат в плоскости самого изображения (вдоль его ширины и высоты).

Ось Z является вектором нормали и направлена перпендикулярно плоскости изображения (от зрителя или к зрителю).

Главная техническая задача разработчика заключается в постоянном преобразовании координат из локального пространства распознанного маркера в мировое пространство игрового движка. Математически трансформация описывается матрицей преобразования, которая содержит три вектора данных: позицию (Translation), вращение (Rotation) и масштаб (Scale).

Получение данных о локальной трансформации физического маркера в мировом пространстве позволяет определить точное местоположение объекта. Однако прямое присвоение этих координат виртуальной модели

часто приводит к визуальным артефактам из-за конфликта ориентаций и масштабов, что требует программной корректировки.

1.3 Проблемы позиционирования и методы корректировки пространственных координат

Практика разработки AR-приложений показывает, что базовые алгоритмы AR-фреймворков предоставляют лишь сырые математические данные о трекинге. Использование этих данных «как есть» неизбежно влечет за собой ряд пространственных проблем. Рассмотрим основные из них и методы их корректировки.

1. Проблема некорректного масштабирования (Scale Discrepancy)

Проблема: Различные программы для 3D-моделирования (Blender, Maya, 3ds Max) и графические движки используют разные единицы измерения (миллиметры, метры, дюймы). Движок Unreal Engine оперирует сантиметрами (1 единица = 1 см), тогда как ARCore по умолчанию ведет расчеты в метрах. В результате импортированная 3D-модель при переносе в AR может оказаться во много раз больше или меньше реального окружения.

Метод корректировки: Применение метода масштабирующего множителя (Scale Multiplier). Разработчик программно задает матрицу масштабирования с коэффициентом S , которая применяется к объекту перед его визуализацией. Это позволяет сжать модель до требуемых физических габаритов, не нарушая её пропорций.

2. Проблема конфликта осей ориентации (Orientation Mismatch)

Проблема: 3D-модели обычно создаются с ориентацией «Z-вверх». При использовании вертикальных изображений в качестве маркеров, локальная ось Z маркера направлена перпендикулярно стене (на зрителя). Если программно совместить координаты маркера и модели без корректировок, 3D-модель «ляжет» на спину, вытягиваясь из стены.

Метод корректировки: Внесение статического смещения ориентации (Rotation Offset). Метод заключается в математическом перемножении текущего кватерниона вращения объекта с корректирующим кватернионом. На практике это означает программный поворот модели на -90 или 180 градусов по заданным осям локально, до её помещения в мировую систему координат. Данная корректировка нивелирует разницу между системой координат 3D-редактора и плоскостью AR-маркера.

3. Проблема множественного инстанцирования (Over-spawning)

Проблема: Система трекинга опрашивает камеру с частотой 30-60 раз в секунду. Если алгоритм запрограммирован просто реагировать на триггер «маркер обнаружен», приложение будет создавать новую копию 3D-модели на каждом кадре. Это приводит к критическому падению производительности мобильного устройства из-за переполнения видеопамяти.

Метод корректировки: Реализация логики контроля состояний (State Validation) с динамическим обновлением координат. Вместо непрерывного создания объектов вводится проверка на существование ссылки в памяти. Если объект еще не создан — происходит инициализация и первичное позиционирование. Если объект уже существует — система перестает выделять память под новые объекты и переключается в режим динамической корректировки координат.

4. Динамическая корректировка при пространственном дрейфе (Dynamic Transform Update)

Проблема: Из-за погрешностей в работе камеры и микроскопических движений рук пользователя, координаты физического маркера в мире постоянно меняются на доли миллиметров. Если поместить виртуальный объект в пространство лишь однажды, со временем он «отстанет» от маркера.

Метод корректировки: Применение функции динамического перезаписывания локаций (Set Actor Transform). При каждом обновлении

кадра программа считывает свежую трансформ-матрицу маркера и принудительно переписывает координаты 3D-модели. Этот метод гарантирует, что 3D-модель будет надежно «приклеена» к физическому изображению, плавно перемещаясь и поворачиваясь синхронно с ним.

Таким образом, создание реалистичного AR-опыта требует не только успешной инициализации сессии трекинга, но и выстраивания комплексной логической цепи. Применение методов корректировки пространственных координат позволяет нивелировать технические ограничения мобильных систем и создать интуитивно понятное и стабильное приложение.

2 Разработка AR-приложения с применением методов корректировки координат

2.1 Выбор и обоснование инструментария

Для реализации прототипа мобильного AR-приложения был выбран игровой движок Unreal Engine 5. Выбор обусловлен мощной встроенной системой визуального программирования Blueprints, которая позволяет реализовывать сложную логику трекинга и корректировки пространственных координат без необходимости написания кода на C++. В качестве подсистемы дополненной реальности для платформы Android использовался фреймворк Google ARCore. ARCore предоставляет нативные API для отслеживания движения, понимания окружения и, что наиболее важно для данного проекта, надежные алгоритмы трекинга изображений.

В качестве целевого устройства использовался смартфон под управлением операционной системы Android. В ходе настройки проекта была решена проблема взаимодействия современных версий Android (API Level 31+) с файловой системой Unreal Engine. Для корректной инициализации AR-сессии без нарушения политик безопасности Scoped Storage (ошибка доступа к ОВВ-кэшу), архитектура сборки была изменена на монолитную: данные приложения были программно инкапсулированы внутрь APK-файла, а системные логи перенаправлены в изолированную директорию.

2.2 Настройка системы Image Tracking и архитектура приложения

Архитектура разработанного приложения базируется на распознавании физического маркера в реальном мире. В качестве целевого объекта была выбрана фотография исторического памятника Ю.А. Гагарину.

Для интеграции маркера в среду Unreal Engine был создан объект класса `ARCandidateImage`. На данном этапе был применен базовый метод калибровки физического пространства: движку была передана реальная физическая ширина маркера в метрической системе. Эта настройка является критически важной, так как именно на основе соотношения пикселей в видеопотоке и заданного физического размера `ARCore` вычисляет дистанцию от камеры до объекта и формирует матрицу трансформации.

Данный кандидат был добавлен в конфигурационный файл AR-сессии (`ARSessionConfig`), переведенный в режим отслеживания мирового пространства (`World Tracking`).

2.3 Программная реализация трансформации координат (от локальных координат маркера к мировым)

Основная логика приложения была реализована в классе `BP_ARPawn`, отвечающем за обработку событий камеры. Для обеспечения плавности AR-опыта алгоритм обработки был привязан к событию обновления кадра (`Event Tick`). На каждом кадре система запрашивает массив всех отслеживаемых AR-геометрий (`Get All AR Geometries`) и фильтрует их, выделяя только объекты типа `ARTrackedImage`.

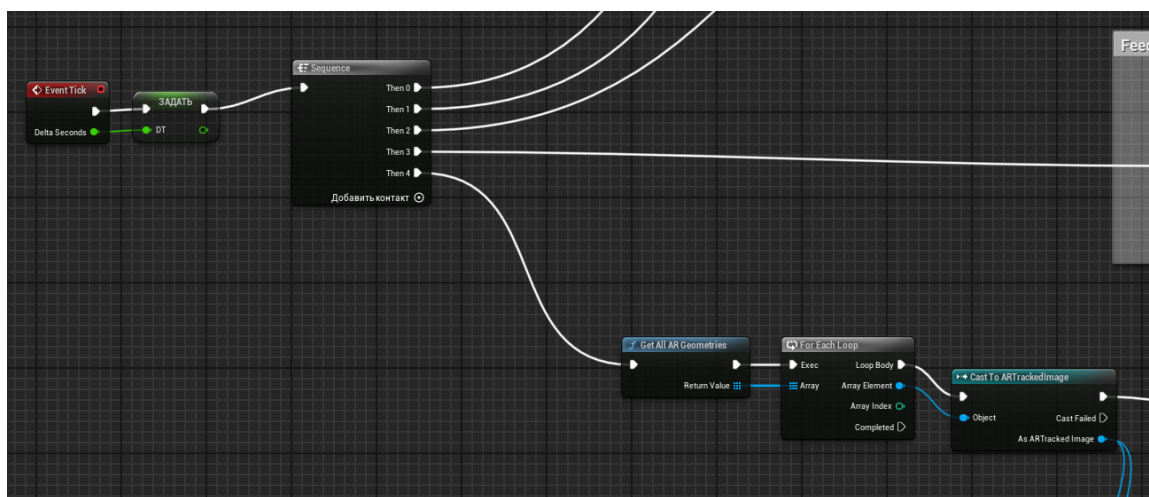


Рисунок 1 – Схема `BP_ARPawn` (`Event Tick`, `Get All AR Geometries`)

Далее алгоритм проверяет состояние трекинга (Get Tracking State). Если статус соответствует значению Tracking (маркер уверенно распознается камерой), инициируется процесс пространственной трансформации.

Во избежание классической проблемы AR-разработки — множественного инстанцирования объектов (Over-spawning) на каждом кадре, что ведет к переполнению памяти и падению частоты кадров, — был применен метод контроля состояний посредством узла валидации (Is Valid).

Логика работает по следующему алгоритму:

1. Если ссылка на виртуальный памятник пуста (Is Not Valid), система вызывает функцию Spawn Actor from Class, создавая 3D-модель в сцене.

2. Координаты для создания извлекаются из функции Get Local to World Transform, которая конвертирует локальные 2D-координаты найденной картинки в 3D-координаты мирового пространства Unreal Engine. Созданный объект записывается в переменную для дальнейшего отслеживания.

3. Если объект уже существует (Is Valid), алгоритм обходит функцию создания и переходит к динамической корректировке.

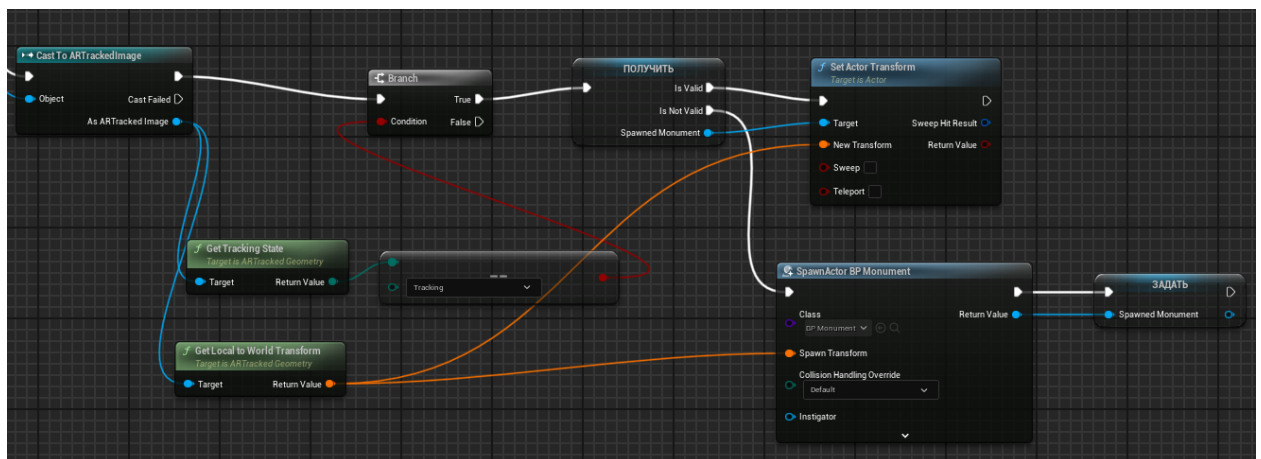


Рисунок 2 – Программная реализация отслеживания маркера и контроля создания объектов (Get Tracking State, Spawn Actor from Class)

2.4 Практическое применение корректировок

После получения сырых координат от маркера к 3D-модели были применены три метода пространственной корректировки для достижения реалистичного визуального результата:

1. Динамическая корректировка позиции. На каждом кадре, если виртуальный памятник уже существует, вызывается функция `Set Actor Transform`. На вход этой функции подается актуальная мировая матрица маркера. Это заставляет 3D-модель непрерывно обновлять свои координаты, жестко «приклеиваясь» к физической картинке даже при дрожании рук пользователя или изменении угла обзора.

2. Статическая корректировка масштаба (`Scale Correction`). Импортированная 3D-модель памятника Ю.А. Гагарину в формате `.glb` имела исходные габариты, несовместимые с рабочим пространством AR. Для решения проблемы был применен метод математического масштабирования. Внутри класса памятника (`BP_Monument`) компонентам модели был задан корректирующий множитель по осям X , Y , Z равный 0.01 . Кроме того, для оптимизации рендеринга на мобильном устройстве (GPU) полигональная сетка модели (`Static Mesh`) была программно упрощена путем настройки уровня детализации (`LOD`), что позволило сократить количество треугольников до 10% от оригинала без заметной потери качества.

3. Корректировка вектора нормали (`Rotation Offset`). Наиболее значимой пространственной корректировкой стало исправление ориентации модели. По умолчанию нормаль Z маркерного изображения направлена перпендикулярно его плоскости (из экрана/стенда). Вследствие этого 3D-модель при спавне визуализировалась горизонтально (лежа на спине). Для приведения локальной системы координат модели в соответствие с визуальными ожиданиями, к компонентам памятника была применена статическая корректировка угла Эйлера: поворот на -90 градусов по оси Y .

Это обеспечило строго вертикальное расположение виртуального объекта относительно вертикального маркера.



Рисунок 3 – Модель памятника

2.5 Разработка UI-интерфейса в контексте AR-сцены

Для повышения информативности приложения поверх дополненной реальности был интегрирован пользовательский интерфейс (UI). Задача заключалась в том, чтобы текстовая информация о памятнике не перекрывала виртуальный объект постоянно, а вызывалась по требованию пользователя.

С помощью инструмента UMG (Unreal Motion Graphics) был разработан виджет WBP_Info. На холсте были размещены два основных элемента: кнопка «Подробнее» и текстовый блок с исторической справкой, поддерживающий автоматический перенос строк для корректного отображения на экранах с различным разрешением.

Текстовый блок был преобразован в программную переменную (Is Variable), а его видимость по умолчанию установлена в состояние Hidden (Скрыто). В графике событий виджета (Event Graph) была реализована логика переключателя на основе оператора FlipFlop. При первом нажатии на кнопку вызывается функция Set Visibility, изменяющая статус текстового блока на Visible, при повторном нажатии — возвращающая его в скрытое состояние.

Виджет инициализируется и добавляется на экран (Add to Viewport) в событии Event BeginPlay внутри класса BP_Monument. Таким образом, интерфейс появляется на экране мобильного устройства строго в тот момент, когда система трекинга успешно распознает маркер и выполняет трансформацию координат для создания виртуального памятника.



Рисунок 4 – Схема WBP_Info

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было проведено комплексное исследование возможностей и особенностей разработки мобильных приложений для систем дополненной реальности. Главный акцент в работе был сделан на изучение и практическое применение методов корректировки пространственных координат виртуальных объектов.

На этапе теоретического анализа были изучены принципы работы систем мобильного трекинга, а также исследована специфика взаимодействия локальных и мировых систем координат в графических движках. Было установлено, что сырые данные, получаемые от AR-сенсоров смартфона, нуждаются в обязательной программной и математической обработке.

Практическая часть исследования увенчалась успешной разработкой функционального прототипа мобильного AR-приложения на базе движка Unreal Engine 5 и фреймворка Google ARCore. В процессе создания приложения были успешно реализованы и протестированы следующие методы пространственной корректировки:

1. Трансформация координат: реализован алгоритм конвертации двумерных локальных координат распознанного маркера в трехмерное мировое пространство движка.

2. Динамическая корректировка позиции: применен метод покадрового обновления матрицы трансформации виртуального объекта, что позволило устранить пространственный дрейф и обеспечить привязку 3D-модели к физическому маркеру в реальном времени.

3. Статическая корректировка вектора нормали и масштаба: программно нивелирована разница между метрической системой 3D-редактора и AR-сцены, а также скомпенсировано несовпадение оси Z маркера с мировой вертикалью.

Кроме того, в ходе работы были успешно решены специфические инженерные задачи мобильной AR-разработки. Разработан алгоритм контроля состояний, предотвращающий проблему множественного инстанцирования объектов, проведена оптимизация полигональной сетки 3D-модели, а архитектура сборки адаптирована под требования безопасности современных версий ОС Android. Интеграция интерактивного пользовательского интерфейса доказала возможность создания удобных информационных систем поверх AR-камеры.

Проделанная работа подтверждает, что применение методов пространственной корректировки является неотъемлемым этапом AR-разработки. Созданный прототип интерактивного гида по историческому памятнику полностью работоспособен и может служить технологическим фундаментом для создания более масштабных образовательных, туристических или музейных приложений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Unreal Engine 5. [Электронный ресурс] // Epic Games Developer. – URL: <https://dev.epicgames.com/documentation/ru-ru/unreal-engine/> (дата обращения: 09.05.2026).
2. Руководство разработчика ARCore. [Электронный ресурс] // Google Developers. – URL: <https://developers.google.com/ar> (дата обращения: 09.05.2026).
3. Документация по работе с дополненной реальностью (AR) в Unreal Engine. [Электронный ресурс] // Epic Games. – URL: <https://dev.epicgames.com/documentation/unreal-engine/augmented-reality-overview-in-unreal-engine> (дата обращения: 09.05.2026).
4. Шикин, Е. В. Компьютерная графика. Полигональные модели / Е. В. Шикин, А. В. Боресков. – Москва: ДМА-Пресс, 2015. – 464 с.
5. Craig, A. B. Understanding Augmented Reality: Concepts and Applications / A. B. Craig. – Waltham, MA: Morgan Kaufmann, 2013. – 296 p.
6. Schmalstieg, D. Augmented Reality: Principles and Practice / D. Schmalstieg, T. Hollerer. – Boston: Addison-Wesley Professional, 2016. – 528 p.
7. Математика в 3D-графике и AR: Кватернионы и матрицы трансформаций. [Электронный ресурс] // Habr. – URL: <https://habr.com/ru/articles/> (дата обращения: 10.05.2026).
8. Оптимизация 3D-моделей для мобильных устройств: уровни детализации (LOD). [Электронный ресурс] // Unreal Engine Documentation. – URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-and-using-lods-in-unreal-engine> (дата обращения: 10.05.2026).
9. Интеграция пользовательских интерфейсов UMG в AR-сцены. [Электронный ресурс] // Epic Games Developer Community. – URL: <https://dev.epicgames.com/community/> (дата обращения: 10.05.2026).

10. Требования к сборке APK для современных версий ОС Android (API Level 31+). [Электронный ресурс] // Android Developers. – URL: <https://developer.android.com/about/versions/13/behavior-changes-all> (дата обращения: 10.05.2026).